TECNOLOGIAS
E ARQUITETURA

GitDash - Software Architecture and Design - Project Report

Master's in Computer Science and Business Management

Gonçalo Gonçalves Miranda

Teacher:
PhD José Pereira dos Reis
Assistant Professor, ISCTE-IUL

ISCTE – Instituto Universitário de Lisboa
January, 2026

Contents

List of Figures iii

List of Tables v

Chapter 1. Introduction 1

Chapter 2. Project Development 3
2.1. Functional Requirements 3
2.1.1. Authentication via GitHub OAuth 3
2.1.2. Selection of a Specific Repository 3
2.1.3. Automatic Identification of Project Contributors 4
2.1.4. Individual Contributor Dashboard with Metrics 4
2.1.5. Graphical Visualizations 4
2.1.6. Responsive and Intuitive User Interface 4
2.2. Solution Architecture 4
2.2.1. High-level Architecture 4
2.2.2. Architecture Design Patterns 4
2.2.2.1. Dependency Injection Pattern 5
2.2.2.2. Singleton Pattern 5
2.2.2.3. Decorator Pattern 6
2.2.2.4. Guard Pattern 6
2.2.2.5. Middleware Pattern 6
2.2.2.6. Facade Pattern 6
2.2.2.7. Mapper Pattern 6
2.2.2.8. Repository Pattern 6
2.2.2.9. Caching Pattern 7
2.2.2.10. Factory Pattern 7
2.2.2.11. Observer Pattern 7
2.2.2.12. Composite Pattern 7
2.2.2.13. Command and Cancellation Pattern 7
2.2.2.14. Debounce Pattern 7
2.2.2.15. Template Pattern 8
2.2.3. Architecture Quality Attributes 8
2.2.3.1. Security and Authentication 8
2.2.3.2. Performance and Scalability 8

2.2.3.3. Reliability and Availability 9
2.2.3.4. Usability and Accessibility 9
2.2.3.5. Maintainability and Extensibility 10
2.2.3.6. Error Handling and Observability 10
2.2.3.7. Data Handling and Privacy 10
2.2.3.8. Deployability and Portability 11
2.2.3.9. Testing and Quality Assurance 11
2.2.4. Solution Trade-offs 11

Chapter 3. Conclusion 13

List of Figures

1 High Level Architecture 5

List of Tables

CHAPTER 1

Introduction

In modern days software development, GitHub plays a central role in coordinating distributed teams and managing complex projects, as the development process is increasingly collaborative and data-driven. While it provides a wide range of analytics and repository insights out of the box, Github often displays them across multiple views and not by highlighting individual contributor activity in a unified, centralized way. As projects grow in size and complexity,it can be benefitial to clearly view into participation patterns, contribution levels, and recent activity.

This project, that was developed for the Software Architecture and Design course at ISCTE-IUL, targets precisely that need by proposing and implementing an API-based user dashboard for GitHub projects, focused on aggregating and visualizing contributor-level metrics in an intuitive, responsive interface. By integrating directly with the GitHub API and using the platform's OAuth, the system is able to securely access repository data and automatically identify all contributors involved in a given project. For each contributor, the application generates a personalized dashboard that presents key indicators about the contributors envolvment in the repository's code development.

The solution emphasizes not only data collection, but also meaningful presentation, as there was also a focuse to graphically present the gathered data in comprehensible ways, providing better insights to the contributors about their performance. This was achieved by using both line charts and pie charts.

CHAPTER 2

Project Development

This section describes how the system was developed, focusing on how the proposed architecture and implementation choices address the project's functional requirements, as well as how it fulfills common quality attributes. The solution was developed as a full-stack web application composed of a frontend client written in Next.js and a backend API, designed with NestJS in order to collect, process, and visualize data from GitHub at both repository and contributor levels. This chapter also details design decisions and some trade-offs made during the development process, in order to achieve a system that fulfills the initially defined functional requirements.

2.1. Functional Requirements

This section goes into how the functional requirements defined for the project were fullfilled by the developed system. All the requirements were implemented by coordinating the client-side application (the frontend) and the server-side API (the backend), where the former is responsible for user interaction and visualization, and the latter manages authentication, data retrieval, aggregation, and integration with the GitHub API.

2.1.1. Authentication via GitHub OAuth

The system supports user authentication through GitHub OAuth 2.0, allowing users to use their GitHub accounts to sign. The authentication flow starts on the frontend, with a user click on the login button, which then initiates a redirection to GitHub's authorization service that, upon successful authentication, redirects the user to one of the backend routes, where an application session is generated and passed on to the frontend as a cookie, that is later used to authenticate subsequent requests. Route protection is implemented on the frontend's middleware layer, at the context layer and on the backend's controller layer, which guarantees that any unauthenticated user is unable to access protected content, and session validity is continuously enforced to prevent unauthorized access and force the users to re-authenticate themselves as soon as possible. It is also possible for a user to terminate their session through a logout mechanism, ensuring proper session lifecycle management.

2.1.2. Selection of a Specific Repository

After authentication, users can select a Github repository from a list of their own repositories, or search public repositories either by entering their URL or name. Once a repository is selected, the frontend maintains the selection consistently across views and uses it as the context for subsequent data retrieval. This mechanism enables seamless navigation between repositories and ensures that all displayed data remain synchronized with the currently selected project.

2.1.3. Automatic Identification of Project Contributors

After the repository selection, all its contributors are dynamically fetched and associated with the project. Contributor data is presented incrementally to support repositories with a large number of contributors. The system maintains an updated list of contributors and uses this data for further exploration and metric generation.

2.1.4. Individual Contributor Dashboard with Metrics

After display all contributors on a specific repository, the system allows the user to select a contributor to explore. When the selection happens, an individual dashboard that presents contributor metrics relevant to project activity and participation is fetched from the backend (which then fetches data from GitHub's APi and processes it into structured, insightful responses), utilizing parallelized API calls in order to smoothen the data loading. These metrics include the number of commits, lines of code added and removed, issues opened and closed, pull requests submitted and approved, recent development activity by week, and PR conversion rate.

2.1.5. Graphical Visualizations

Graphical visualizations is supported by the system, enabling intuitive exploration of contribution data, via the utilization of line charts and pie charts. This enables users to identify trends, distributions, and relative contributions at a glance. Visualizations are generated from aggregated data and are dynamically updated according to repository and/or contributor context changes.

2.1.6. Responsive and Intuitive User Interface

The user interface is designed to be responsive and intuitive, being easy and intuitive to interact with and adaptable to different screen sizes. Navigation, search, and dashboard interactions are designed to support efficient exploration of repository data. The UI elements are structured to provide clear feedback and maintain usability across devices, ensuring that contribution insights remain accessible to a wide a variety of users and usage scenarios.

2.2. Solution Architecture

2.2.1. High-level Architecture

The system follows a client-server architecture composed, of a web frontend application designed with Next.js and a backend API, with GitHub's APIs acting as the external data source. At a high level (as illustrated in **Figure 1**), the frontend is responsible for authentication initiation, navigation, and the visualization of data, while the backend is used to process authentication logic, data collection, aggregation, caching, and response shaping. Redis is also used as a cache for analytics data and for session data.

2.2.2. Architecture Design Patterns

This project leverages several well-established design patterns across both the backend and the frontend components, in order to improve modularity, maintainability, scalability, and clarity of

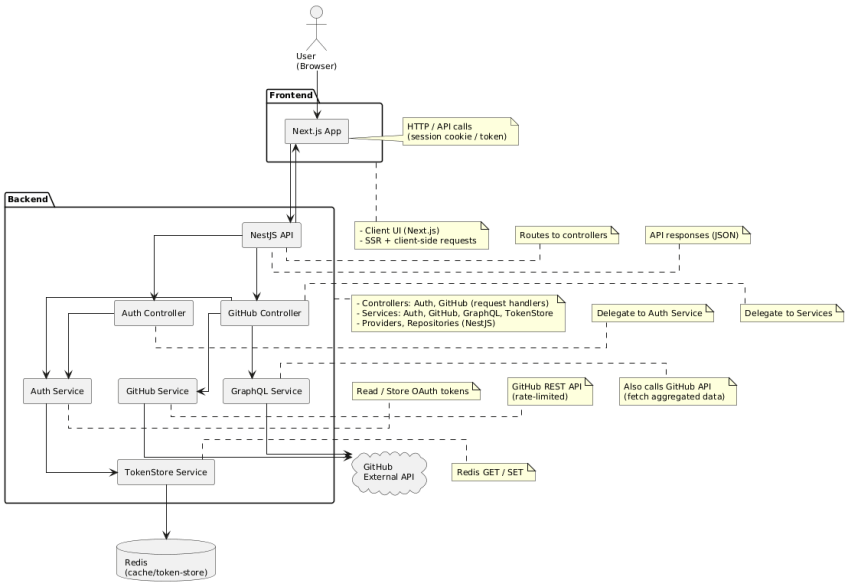

Figure 1. High Level Architecture

responsibility. Rather than just being applied individually, these patterns naturally emerge from the architectural choices made — particularly the use of the frameworks in use.

The backend architecture heavily benefits from NestJS's opinionated design, which encourages the use of proven enterprise patterns, while The frontend design patterns applied to the frontend are the natural outcome of using Next.js (and React, its base), focusing on state management, UI composition, and interaction control.

2.2.2.1. Dependency Injection Pattern

The Dependency Injection pattern is applied through NestJS's `@Injectable()` mechanism and constructor-based injection. Core services, including authentication, GitHub integration, GraphQL Integration, token storage, and data mapping, are injected into dependent components rather than being instantiated directly, which decouples components from concrete implementations, enhanced testability by enabling mocking or substitution of dependencies, and improves maintainability by localizing changes to specific services without affecting dependent modules.

2.2.2.2. Singleton Pattern

On the backend, NestJS providers are instantiated as singletons by default within the application context. As a result, all the services that are injected are only created once and reused throughout the application lifecycle. Due to this behaviour, shared resources, such as Redis connection, are consistently and efficiently managed, reducing instantiation overhead and maintaining a single, coherent state across requests.

This pattern is, however, also used on the frontend to control global application state, by instantiation a single React Context provider, which ensures consistent data access across components and aids in avoiding prop drilling.

2.2.2.3. Decorator Pattern

NestJS's decorator-based programming model is extensively used, as built-in decorators such as @Controller, @Get, and @UseGuards simplify request routing and security enforcement. Additionally, a custom decorators can be created and, in this case, one is used to extract session identifiers from JWT payloads, reducing boilerplate code inside controllers and improving readability and maintainability.

2.2.2.4. Guard Pattern

The Guard pattern is applied by the backend to enforce authorization rules on protected resources, using the @UseGuard decorator. A custom JWT-based guard intercepts incoming requests and evaluates whether or not the requesting client is authorized before allowing access to controller endpoints, which enables the usage of access control policies consistently across the application and helps avoid duplicate checks within individual controllers.

2.2.2.5. Middleware Pattern

Middleware is used by the frontend (almost as a guard) at the application level to handle cross-cutting concerns that apply to all incoming requests before they are processed by any route. These concerns include request preprocessing tasks such as CORS configuration, cookie parsing, and session-related handling, which also aids the clear separation between infrastructural concerns and business logic, improving maintainability and readability.

2.2.2.6. Facade Pattern

Both the main services used by the backend, such as the GitHub service and authentication service act as facades, as they provide simplified, high-level methods that encapsulate complex logic involving multiple API calls, data transformations, and caching steps. Controllers can be abstracted from business logic by interacting with these services through concise method calls, without needing to understand the underlying complexity of GitHub APIs.

2.2.2.7. Mapper Pattern

On the backend, the dedicated mapper service transforms raw GitHub API responses into internal domain models used by the application, which isolates the system from changes in external API schemas and ensures the stability of the date structures the internal application logic interacts with, while also keeping transformation logic centralized.

2.2.2.8. Repository Pattern

The backend's token store service applies a Repository-like pattern by abstracting persistence operations related to sessions, OAuth state, and cached repository statistics by exposing domain-specific methods for creating, retrieving, and deleting stored data, being the only point of direct interaction with the underlying storage mechanism (in this case, Redis) within the whole application. This abstraction improves maintainability and allows the persistence layer to be modified or replaced without impacting higher-level services.

2.2.2.9. Caching Pattern

Caching is implemented on the backend using Redis with configurable time-to-live (TTL) values to temporarily store session data and computed GitHub statistics (which, for bigger repositories, is a call that can last for a few seconds, which isn't ideal). This follows the Caching pattern by reducing redundant external API calls and improving response times. Cached entries are automatically expired by Redis, which ensures data freshness while maintaining efficient resource usage.

2.2.2.10. Factory Pattern

The Redis client on the backend is instantiated using a factory function that centralizes configuration concerns such as connection options, retry strategies, and event registration, which allows the application to ensure consistent Redis client setup and simplifies future modifications to the persistence configuration without affecting dependant services.

2.2.2.11. Observer Pattern

The application's frontend implements the observer pattern in multiple forms, ranging from a custom hook that periodically observes session validity and updates global state accordingly, which allows the UI to react to authentication changes, to the usage of the IntersectionObserver API in order to monitor scrolling behavior on the contributors navbar and trigger incremental loading of contributors, enabling efficient infinite scrolling.

2.2.2.12. Composite Pattern

The individual UI components on the application's frontend are composed hierarchically into higher-level structures such as dashboards, summary panels, and navigation layouts and, ultimately, into pages. This composition enables complex interfaces to be designed from simpler building blocks, allowing consistent behavior across pages while maintaining flexibility in the generation of new pages.

2.2.2.13. Command and Cancellation Pattern

Network requests executed by the frontend are treated as cancellable commands through the use of abort controllers, as this allows in-flight requests to be explicitly cancelled when they become obsolete, such as during rapid search input changes. By controlling a request's lifecycle, processing stale responses can be avoided and improves both performance and data correctness.

2.2.2.14. Debounce Pattern

The application's frontend uses debounce mechanisms in user interactions, such as window resize events and search input handling. By delaying execution until user activity stabilizes, the application reduces unnecessary re-renders and network requests, leading to a better user experience and to improvals on resource usage efficiency.

2.2.2.15. Template Pattern

Reusable UI Components, such as buttons, inputs, and dialog components (some from ShadCN's library), follow a template-style approach that defines the component's structure, behavior, and styling throughout the application in a consistent manner. By encapsulating common interaction patterns within these primitives, the frontend is able to enforce an uniform user experience and reduces boilerplate, improving maintainability and UI extensibility.

2.2.3. Architecture Quality Attributes

In addition to the system's functional requirements, the proposed architecture was designed to support a set of key software quality attributes that contribute to the system's robustness, extensibility, and long-term sustainability. These attributes are aligned with widely accepted quality attributes, such as security, performance, usability, maintainability, and deployability. The following sections explain which attributes are supported by the current architecture and how they were implemented.

2.2.3.1. Security and Authentication

Security is a critical quality attribute for most systems, especially if third-party platforms are integrated or if user identity data is handled. The application implements secure authentication and session management across both the frontend and the backend.

On the frontend, authentication is handled through the GitHub OAuth flow. The login component redirects users to a configured authentication endpoint (on Next.js' server side routes, where it then re-directs the user to Github's OAuth flow, which finally redirects the users to a route on the backend which redirects to the main dashboard route on the frontend), while a middleware enforces server-side route protection, that validates sessions against the backend API. Session cookies are handled consistently being included in all requests made credential-inclusive, and logout operations explicitly trigger the backend route to invalidate the server-side session. Client-side session monitoring is implemented through a dedicated session watcher hook, which periodically validates session state and proactively handles session expiration.

On the backend, OAuth 2.0 is used to delegate authentication to GitHub, eliminating the need to store user credentials locally. Sessions are managed using signed JWTs with configurable expiration times, stored and validated through Redis, while secure cookie settings (HTTP-only, SameSite policies, and TLS support) mitigate common web vulnerabilities, such as XSS and CSRF.

The combination of the mechanisms implemented between the frontend and the backend promotes strong security for the system.

2.2.3.2. Performance and Scalability

When dealing with repositories that may have a large ammount of contributors and, therefore, a huge quantity of data associated with them, performance and scalability are essential quality attributes, as the system must be able to proccess these use-cases as well as small repositories.

On the frontend, responsiveness is achieved through a combination of debounced UI hooks, responsive, external layout components, and efficient rendering strategies. Charts are rendered using responsive containers, and layout adjustments rely on debounced window size detection to avoid unnecessary reflows. Data fetching is optimized through parallel requests using typescript's `Promise.All`, in-flight request cancellation via AbortController, and debounced search inputs, while the contributor list supports infinite scrolling with pagination through the use of the IntersectionObserver API, enabling efficient handling of large datasets.

On the backend, the usage of Redis as a cache to store session data and computed repository statistics are cached with configurable TTLs, results in a smaller number of repeated calls to the GitHub API (which also helps the system avoid getting the rate-limited by Github). Pagination is implemented for API responses such as contributor lists, allowing the system to scale to repositories with large contributor bases, while still supporting smaller repositories. Besides this, GitHub's GraphQL API was also used so that only required data fields were fetched, reducing payload sizes and improving response times substantially. Besides this, parallel computing was also used in order to perform concurrent API requests, which significantly reduces latency when multiple resources are required.

Due to these decisions, the system is responsive under increased load and scales effectively with repository size.

2.2.3.3. Reliability and Availability

A system's reliability is defined as the its ability to correctly and continuosly operate, regardless of the existence of partial failures. This is the one of the core characteristics that was taken into consideration when developing this specific system. Redis connections are configured with keep-alive mechanisms, health checks, and retry strategies using exponential backoff, helping to maintain session availability during transient network issues while API interactions with GitHub are wrapped in structured error handling logic, allowing partial data to be returned even when some requests fail, rather than causing complete request failures. Besides this, session expiration and validation mechanisms are also implemented, in order to make the system more reliable, by ensuring that stale or invalid sessions are detected and gracefully handled. Basic logging of infrastructure events, such as Redis connection state changes, provides operational visibility and facilitates the diagnosing of runtime issues.

2.2.3.4. Usability and Accessibility

As for the system's accessibility, certain ShadCN components were used in for the UI, which provide out of the box accessibility support, such as focus-visible styling and basic ARIA attributes, such as aria-invalid handling in form elements. Alongside this, images on the custom components include alternative text to support assistive technologies. Regarding the usability, a user-friendly UI was developed using both ShadCN and custom components, as well as debounced input handling for smoother interactions was implemented. Even though usability was something that was focused and achieved on this system, accessibility is only partially achieved, as not all the components utilized support assistive technologies.

2.2.3.5. Maintainability and Extensibility

Systems that are envisioned to evolve over time must be maintainable and extensible. To achieve these characteristics, both the frontend and the backend codebases are written entirely with TypeScript as the code-base's main language, providing strong type safety and reducing runtime errors. The frontend was was built to be modular and component-based and the principal of the separation of concerns was followed, including reusable UI primitives, chart components, and a typed global state managed through React context. This structure, alongside the usage of Next.js, which improves maintainability and extensibility through its opinionated project structure and built-in routing and rendering conventions, reduces architectural complexity and allows new pages, components, metrics, charts, or dashboard sections to be added with minimal impact on existing code, further improving the maintainability and extensibility of the system.

As for the backend, it adopts a service-oriented architecture built on NestJS, a framework that leverages dependency injection to promote loose coupling and testability. The main features are separated into dedicated services, improving readability and facilitating extensions. Besides this, configuration is fully externalized through environment variables, enabling changes across environments without code modifications.

2.2.3.6. Error Handling and Observability

As for the error handling and observability, the frontend includes basic validation of responses and error propagation for network requests with fallback behaviors in place to prevent UI crashes. However, there is no integrated monitoring, structured logging, or external error-reporting service.

On the backend, API calls are protected with try-catch blocks and defensive logic to prevent cascading failures. Events on the infraestructure level, such as the ones related to Redis connectivity, are logged to provide insight into system health.

While this approach is sufficient for a small project, observability could be improved through structured logging and centralized monitoring in a production setting.

2.2.3.7. Data Handling and Privacy

The system also looked to preserve data privacy by minimizing the storage and exposure of personal data. The frontend stores user and session data in memory through the global context and does not persist it locally beyond the cookies required for authentication, which are also used for session management. The content of this cookie is a JWT token, which only contains a session and user id, the authenticated user's username and the token issue and expiration date. Besides that, session-sensitive requests explicitly disable caching and consistently include credentials to ensure correctness and privacy.

As for the backend, sensitive data such as access tokens and session information are stored securely in Redis and are automaticaly expirated, according to the TTL set to each entry. No unnecessary data is persisted beyond what is required for authentication, aligning with data minimization principles.

2.2.3.8. Deployability and Portability

Reliable builds and deployments across environments are ensured by deployability and portability.

As Next.js is the framework used for the frontend, which supports environment-driven configuration, it is compatible with standard CI/CD pipelines and modern hosting platforms. In this case, the frontend is hosted as a Cloudflare Worker on https://ads-iscte-frontend.goncalo-miranda-cplx.workers.dev.

As for the backend, it was built on NestJS, which is a production-ready Node.js framework designed for scalable server-side applications. Dependency management is handled via pnpm with lock files, ensuring reproducible builds across environments. Environment-based configuration is also supported, which eases deployments. The backend is currently hosted on Render, on https://ads-iscte-backend.onrender.com.

2.2.3.9. Testing and Quality Assurance

Testing is an important quality attribute related to reliability and maintainability. The backend includes a configured Jest testing infrastructure with support for unit and end-to-end tests. The frontend can also easily utilize an external library for such purpose. However, neither the frontend nor the backend implement unit, integration or end-to-end tests.

While testing is not comprehensively implemented, the existing architecture supports the introduction of automated testing frameworks without major refactoring, representing a clear path for future quality improvements.

2.2.4. Solution Trade-offs

Several trade-offs were made during development to balance accuracy, performance, and implementation complexity, being the most notable example the calculation of repository member counts. As GitHub distinguishes between organization members, collaborators, and contributors, and not all contributors necessarily have associated GitHub user accounts. To ensure the data regarding member count and actual members is consistent, anonymous contributors are not taken into consideration when counting the members. This simplification leads to a difference in contributor count between Gitdash's member count and Github's, as the latter also takes into consideration anonymous users, which can't be fetched via API in order to get the statistics. Taking these users into consideration would break the app, as they have no statistics associated in Github's API. This trade-off was considered acceptable given the project's primary focus on comparative contribution patterns rather than exhaustive contributor enumeration, while ensuring predictable behavior across both small and big repositories.

CHAPTER 3

Conclusion

This project successfully delivered a full-stack system for analyzing and visualizing GitHub repository and contributor activity. By combining a modular frontend architecture with a scalable backend service layer, the solution provides an effective platform for exploring activity in a structured and user-friendly manner.

From a technical standpoint, the project successfully integrates modern web technologies with third-party APIs, highlighting challenges related to data aggregation, pagination, and rate-limiting. The architectural decisions adopted throughout the project support several quality attributes, adding up to the system's quality.

Overall, the system meets its defined objectives and provides a solid foundation for any possible future work in repository analytics and developer productivity assessment. It also reinforces the importance of clear architectural separation and informed trade-offs when designing data-intensive web applications.